

第六套题解

目录

T1 最多求余 · T2 各乘一个 · T3 三元组 · T4 切割

T1 最多求余

部分分（累计 20 分）：所有余数相同

所有数对 k 取余后都相同，因此答案就是 $a_1 \bmod k$ 。保存第一个数的余数，读完其余输入后输出即可。时间复杂度为 $O(n)$ ，额外空间复杂度为 $O(1)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, k, a;
    cin >> n >> k;
    cin >> a;
    int ans = a % k;
    for (int i = 2; i <= n; i++) {
        cin >> a;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（累计 60 分）：出现次数两两不同

用 $\text{cnt}[r]$ 记录余数 r 已经出现的次数。每读入一个数，就增加对应计数；如果新次数严格超过当前记录的最大次数，就更新答案。

本档保证实际出现的各余数频数两两不同，所以最终出现最多的余数是唯一的。当它最后一次出现时，其计数一定超过其他余数的最终计数，答案就会被更新为它。

这里没有处理并列：如果两种余数最终次数相同，较大的余数可能更早达到该次数，从而被保留下来。

时间复杂度为 $O(n)$ ，空间复杂度为 $O(k)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int cnt[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, k;
    cin >> n >> k;
    int ans = 0, best = 0;
    for (int i = 1; i <= n; i++) {
        int a;
        cin >> a;
        int r = a % k;
        cnt[r]++;
        if (cnt[r] > best) {
            best = cnt[r];
            ans = r;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

```
// 本程序适用于最高出现次数唯一的情况，不处理并列最优。
for (int i = 1; i <= n; i++) {
    int x;
    cin >> x;
    int r = x % k;
    cnt[r]++;
    if (cnt[r] > best) {
        best = cnt[r];
        ans = r;
    }
}
cout << ans << '\n';
return 0;
}
```

满分做法：计数后从小到大扫描

对 k 取余的结果只可能在 $[0, k - 1]$ 中。用 $\text{cnt}[r]$ 记录余数 r 的出现次数，读入每个数时执行 $\text{cnt}[a[i] \% k]++$ 。

统计完成后，令 $\text{ans}=0$ ，按 $r = 1, 2, \dots, k - 1$ 的顺序检查各个余数。只有当 $\text{cnt}[r] > \text{cnt}[\text{ans}]$ 时，才令 $\text{ans}=r$ 。

这样既能找到最大的出现次数，也能处理并列：由于从小到大扫描，次数相等时不更新，保留下来的就是最小余数。

注意余数 0 也可能是答案；扫描上界取决于 k ，而不是输入个数 n 。例如 n 很小、 k 很大时，获胜余数仍然可能大于 n 。

时间复杂度为 $O(n + k)$ ，空间复杂度为 $O(k)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

int cnt[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, k;
    cin >> n >> k;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        cnt[x % k]++;
    }
    int ans = 0;
    // 从小到大扫描；只有次数严格更多时才更新。
    for (int r = 1; r < k; r++) {
        if (cnt[r] > cnt[ans]) {
            ans = r;
        }
    }
    cout << ans << '\n';
    return 0;
}
```

T2 各乘一个

以下四组条件各独立占 20 分，不能把标题中的分值理解为累计分。

部分分（独立 20 分， $n \leq 20$ ）：枚举下标组合

一个方案由递增下标 $i_1 < i_2 < \dots < i_x$ 唯一确定。选出第 j 个位置后，它就与 b_j 配对，不需要再枚举系数的排列。

用 DFS 维护下一个可选位置 **start**、已经选择的数量 **chosen** 和当前分数 **sum**。选择位置 i 后，递归到 $i + 1$ ，并把 $a_i b_{chosen+1}$ 加入分数。恰好选满 x 项时更新答案。

还需要选择 $x - chosen$ 项时，下一位置最多取到 $n - (x - chosen) + 1$ ，否则后面剩余的位置不够。初始答案应为负无穷，因为恰好选满后的最大分数也可能是负数。

时间复杂度上界为 $O(x \binom{n}{x})$ ；存储输入及递归栈的空间复杂度为 $O(n + x)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const long long NEG = -(1LL << 60);
int n, x, a[100005], b[11];
long long ans = NEG;
void dfs(int start, int chosen, long long sum) {
    if (chosen == x) {
        ans = max(ans, sum);
        return;
    }
    for (int i = start; i <= n - (x - chosen) + 1; i++) {
        dfs(i + 1, chosen + 1, sum + 1LL * a[i] * b[chosen + 1]);
    }
}
int main() {
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
    }
    dfs(1, 0, 0);
    cout << ans << '\n';
    return 0;
}
```

部分分（独立 20 分， $x = 1$ ）：比较单项乘积

只需要选一个位置，答案就是所有 $a_i b_1$ 的最大值。扫描数组，逐个计算乘积并更新答案。

应当比较乘积，而不是直接取最大的 a_i 。当 $b_1 < 0$ 时，较小的 a_i 反而可能得到较大的分数。

时间复杂度为 $O(n)$ ；下面的代码保存输入数组，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const long long NEG = -(1LL << 60);
int n, x, a[100005], b[11];
long long ans = NEG;
int main() {
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
    }
    for (int i = 1; i <= n; i++) {
        ans = max(ans, 1LL * a[i] * b[1]);
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（独立 20 分， $x = 2$ ）：维护前缀最优

固定第二个位置 j ，分数为

$$a_i b_1 + a_j b_2, \quad i < j.$$

第二项已经确定，左边只需要找到最大的 $a_i b_1$ 。从左到右枚举 j ，用 **left** 维护位置 $1 \sim j-1$ 中最大的 $a_i b_1$ ，再用 **left + a[j] * b[2]** 更新答案。

必须先计算以 j 为第二个位置的方案，再把 $a_j b_1$ 加入前缀最优，避免把同一个位置使用两次。

时间复杂度为 $O(n)$ ；保存输入数组时，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const long long NEG = -(1LL << 60);
int n, x, a[100005], b[11];
long long ans = NEG;
int main() {
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
    }
    long long left = 1LL * a[1] * b[1];
    for (int j = 2; j <= n; j++) {
        ans = max(ans, left + 1LL * a[j] * b[2]);
        left = max(left, 1LL * a[j] * b[1]);
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（独立 20 分， $x = 3$ ）：固定中间位置

固定第二个位置 j 后，左边的第一个位置和右边的第三个位置互不影响，可以分别取最优。

预处理 $\text{leftBest}[i]$ ，表示前 i 项中最大的 $a_i b_1$ ；再预处理 $\text{rightBest}[i]$ ，表示从位置 i 开始的后缀中最大的 $a_i b_3$ 。

枚举 $j = 2, \dots, n - 1$ ，该中间位置对应的最大分数为

$$\text{leftBest}[j - 1] + a_j b_2 + \text{rightBest}[j + 1].$$

左右分别使用 $j - 1$ 和 $j + 1$ ，保证三个位置严格递增。比较所有中间位置的结果即可。

时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const long long NEG = -(1LL << 60);
int n, x, a[100005], b[11];
long long ans = NEG;
long long leftBest[100005], rightBest[100005];
int main() {
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
    }
    leftBest[0] = NEG;
    rightBest[n + 1] = NEG;
    for (int i = 1; i <= n; i++) {
        leftBest[i] = max(leftBest[i - 1], 1LL * a[i] * b[1]);
    }
    for (int i = n; i >= 1; i--) {
        rightBest[i] = max(rightBest[i + 1], 1LL * a[i] * b[3]);
    }
    for (int j = 2; j < n; j++) {
        long long value = leftBest[j - 1] + 1LL * a[j] * b[2];
        value += rightBest[j + 1];
        ans = max(ans, value);
    }
    cout << ans << '\n';
    return 0;
}
```

部分分综合做法

将上述做法合并： $x = 1$ 时取单项最大值， $x = 2$ 时使用前缀最优， $x = 3$ 时使用前后缀最优；其他情况使用 DFS 枚举下标组合。

这份程序可以处理 $n \leq 20$ 的小数据，也可以处理 $x \leq 3$ 的大数组。四组条件各独立占 20 分，合计覆盖 80 分。

$x \leq 3$ 时，时间复杂度为 $O(n)$ ；其他情况的时间复杂度上界为 $O(n + x \binom{n}{x})$ 。空间复杂度为 $O(n + x)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const long long NEG = -(1LL << 60);
int n, x, a[100005], b[11];
long long leftBest[100005], rightBest[100005], ans = NEG;

// 其余情况枚举所选下标；没有故意输出错误值的范围判断。
void dfs(int start, int chosen, long long sum) {
    if (chosen == x) {
        ans = max(ans, sum);
        return;
    }
    for (int i = start; i <= n - (x - chosen) + 1; i++) {
        dfs(i + 1, chosen + 1, sum + 1LL * a[i] * b[chosen + 1]);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
    }
    leftBest[0] = NEG;
    for (int i = 1; i <= n; i++) {
        leftBest[i] = max(leftBest[i - 1], 1LL * a[i] * b[1]);
    }
    if (x == 1) {
        ans = leftBest[n];
    } else if (x == 2) {
        for (int j = 2; j <= n; j++) {
            ans = max(ans, leftBest[j - 1] + 1LL * a[j] * b[2]);
        }
    } else if (x == 3) {
        rightBest[n + 1] = NEG;
        for (int i = n; i >= 1; i--) {
            rightBest[i] = max(rightBest[i + 1], 1LL * a[i] * b[3]);
        }
        for (int j = 2; j < n; j++) {
            ans = max(ans, leftBest[j - 1] + 1LL * a[j] * b[2] + rightBest[j + 1]);
        }
    } else {
        dfs(1, 0, 0);
    }
    cout << ans << '\n';
    return 0;
}
```

满分做法：前缀动态规划

令 $dp[i][j]$ 表示：只考虑 a 的前 i 项，恰好选出 j 项，按顺序与 b 的前 j 项配对时，能够得到的最大分数。

处理位置 i 时只有两种选择。不选 a_i ，答案来自 $dp[i - 1][j]$ ；选择 a_i ，它必须作为第 j 个选中的数，贡献为 $a_i b_j$ ，前面还需要恰好选出 $j - 1$ 项。因此

$$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - 1] + a_i b_j).$$

两种情况包含了所有合法方案，而且第二种只从上一行转移，不会重复使用位置 i 。

初始化 $dp[i][0] = 0$ 。空前缀不能选出正数个元素，所以 $dp[0][j] = -\infty (j > 0)$ ；不可达状态不参加加法。不能把所有状态初始化为 0，否则负分方案可能被不存在的空方案替代。

按 i 从小到大计算，最后输出 $dp[n][x]$ 。数组 a 和 b 都不能排序，因为配对顺序是题目条件。乘法需要先转为 `long long`，例如 `1LL * a[i] * b[j]`。

时间复杂度为 $O(nx)$ ，空间复杂度为 $O(nx)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const long long NEG = -(1LL << 60);
long long dp[100005][11];
int a[100005], b[11];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, x;
    cin >> n >> x;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for (int j = 1; j <= x; j++) {
        cin >> b[j];
        dp[0][j] = NEG;
    }
    // dp[i][j]: 前 i 个元素中恰好选 j 个，与 b 的前 j 项配对。
    for (int i = 1; i <= n; i++) {
        dp[i][0] = 0;
        for (int j = 1; j <= x; j++) {
            dp[i][j] = dp[i - 1][j];
            if (dp[i - 1][j - 1] != NEG) {
                long long value = dp[i - 1][j - 1] + 1LL * a[i] * b[j];
                dp[i][j] = max(dp[i][j], value);
            }
        }
    }
    cout << dp[n][x] << '\n';
    return 0;
}
```

T3 三元组

部分分（累计 20 分）：小规模直接枚举

本档 $n \leq 1000$ ，并且取模前的总价值小于 $10^9 + 7$ 。直接枚举 $i = 1, \dots, n$ 和 $j = i, \dots, n$ ，计算 $A = a_i + a_j$ 、 $B = b_i + b_j$ 。

如果 $2 \min(A, B) \leq \max(A, B)$ ，就把 $c_i c_j$ 加入答案。 j 从 i 开始，既保留了自身配对，也不会把 (i, j) 和 (j, i) 重复计算。

总价值已保证小于模数，因此无需取模；但坐标相加、乘二及权值相乘仍要使用 `long long`。

时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;
long long a[100005], b[100005], c[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i] >> b[i] >> c[i];
    }
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        // j 从 i 开始：允许选择同一个三元组两次。
        for (int j = i; j <= n; j++) {
            long long x = a[i] + a[j];
            long long y = b[i] + b[j];
            if (2 * min(x, y) <= max(x, y)) {
                ans += c[i] * c[j];
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

部分分（累计 40 分， $n \leq 1000$ ）：枚举并取模

枚举方式与上一档相同，但总价值不再有较小上界。每得到一个合法贡献，就对答案取模：

$$ans = (ans + c_i c_j) \bmod (10^9 + 7).$$

加法和乘法可以逐步取模，不必先保存完整总和。单个乘积仍使用 `long long`，避免在取模前溢出。

时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;
long long a[100005], b[100005], c[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i] >> b[i] >> c[i];
    }
    long long ans = 0;
    for (int i = 1; i <= n; i++) {
        // j 从 i 开始: 允许选择同一个三元组两次。
        for (int j = i; j <= n; j++) {
            long long x = a[i] + a[j];
            long long y = b[i] + b[j];
            if (2 * min(x, y) <= max(x, y)) {
                ans = (ans + c[i] * c[j]) % MOD;
            }
        }
    }
    cout << ans << '\n';
    return 0;
}
```

满分做法：拆分条件后排序计数

记 $A = a_i + a_j$ 、 $B = b_i + b_j$ ，则条件可以拆成

$$2A \leq B \quad \text{或} \quad 2B \leq A.$$

因为 $A, B > 0$ ，两种情况不可能同时成立：如果同时成立，就会有 $4A \leq 2B \leq A$ ，与 $A > 0$ 矛盾。因此可以分别统计两种情况，然后相加。

先考虑 $2A \leq B$ 。令 $x_i = 2a_i - b_i$ ，条件就变成

$$x_i + x_j \leq 0.$$

把每个 x_i 与它的权值 c_i 一起按 x_i 升序排序。设 $pre[t]$ 为排序后前 t 个权值之和，对模数取模。

从左到右枚举位置 l ，用 r 表示最大的合法右端点。当 $x_l + x_r > 0$ 时，就把 r 向左移动。随着 l 增大， x_l 不会变小，所以 r 只会左移。

若 $r \geq l$ ，与位置 l 配对且排序后位置不小于 l 的合法元素恰好是 $[l, r]$ ，本次贡献为

$$c_l(pre[r] - pre[l-1]).$$

区间包含 $j = l$ ，所以保留了自身配对；其余无序对只由排序后较靠左的位置统计一次。若 $r < l$ ，后面不再有合法对，可以结束。

再用 $x_i = 2b_i - a_i$ 统计另一种情况，两次结果相加取模即可。前缀和相减时先加模数；键值、键值之和与乘积均使用 `long long`。

时间复杂度为 $O(n \log n)$ ，空间复杂度为 $O(n)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007;
struct Node {
    long long key, weight;
};
Node p[100005];
long long a[100005], b[100005], c[100005], prefix[100005];
int n;

bool cmp(const Node &u, const Node &v) {
    return u.key < v.key;
}

long long calc() {
    sort(p + 1, p + n + 1, cmp);
    prefix[0] = 0;
    for (int i = 1; i <= n; i++) {
        prefix[i] = (prefix[i - 1] + p[i].weight) % MOD;
    }
    long long result = 0;
    int r = n;
    for (int l = 1; l <= n; l++) {
        while (r >= l && p[l].key + p[r].key > 0) {
            r--;
        }
        if (r < l) {
            break;
        }
        // 区间 [l,r] 包含自身，恰好统计一次无序点对。
        long long sum = (prefix[r] - prefix[l - 1] + MOD) % MOD;
        result = (result + p[l].weight * sum) % MOD;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i] >> b[i] >> c[i];
        p[i].key = 2 * a[i] - b[i];
        p[i].weight = c[i];
    }
    long long ans = calc();
    for (int i = 1; i <= n; i++) {
        p[i].key = 2 * b[i] - a[i];
        p[i].weight = c[i];
    }
    ans = (ans + calc()) % MOD;
    cout << ans << '\n';
    return 0;
}
```

T4 切割

部分分（累计 20 分， $n, k \leq 10$ ）：枚举切法

一个 L 位数有 $L - 1$ 个相邻数位间隙，每个间隙只有“切”或“不切”两种选择，因此共有 2^{L-1} 种切法。先单独处理一个数。用二进制掩码表示选中了哪些切点，从左到右扫描数位，得到各段的数值之和及切割次数。只保存和对 k 的余数，用 `best[r]` 记录这个数切开后总和余数为 r 时，最少需要切几次。再合并不同原数。令 `dp[r]` 表示已经处理过的原数，其全部数段之和模 k 为 r 时的最少切割次数。把新数的余数 t 加入后，有

$$\text{nextDp}[(r + t) \bmod k] = \min(\text{nextDp}[(r + t) \bmod k], \text{dp}[r] + \text{best}[t]).$$

初始只有 `dp[0]=0` 可达。每次用独立的 `nextDp` 合并，避免重复使用当前原数。每个原数的切法相互独立，合并时同时累加余数和切割数即可。全部处理后查看 `dp[0]`，不可达时输出 -1 。

设第 t 个数有 L_t 位，时间复杂度为 $O(\sum_t L_t 2^{L_t-1} + nk^2)$ ，额外空间复杂度为 $O(k + \max L_t)$ 。

参考代码

完整代码见下一节“部分分（累计 60 分）：短数字的切法枚举”，两档使用同一程序。

部分分（累计 60 分）：短数字的切法枚举

本档保证 $n, k \leq 100$ 、 $a_i \leq 10^5$ 。每个原数最多 6 位，只有 5 个间隙，最多枚举 32 种切法，因此仍可使用上一节的切法枚举与余数合并。

计算每一段时，可以始终维护 `value = (value * 10 + digit) % k`，不必保存完整段值。到达被选中的切点或当前原数的末尾时，把 `value` 加入段值总和，再将它清零。只有内部切点增加切割次数，原数末尾不增加。

掩码为 0 就表示完全不切；数段以零开头也要正常保留。例如 `100` 可以切成 `1` 和 `00`，后者贡献 0。

时间复杂度仍为 $O(\sum_t L_t 2^{L_t-1} + nk^2)$ ，额外空间复杂度为 $O(k + \max L_t)$ 。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;
int best[405], dp[405], nextDp[405];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, k;
    cin >> n >> k;
    for (int r = 1; r < k; r++) {
        dp[r] = INF;
    }
    for (int i = 1; i <= n; i++) {
        string s;
```

```

cin >> s;
int length = (int)s.size();
for (int r = 0; r < k; r++) {
    best[r] = INF;
    nextDp[r] = INF;
}
// 每个相邻数位间隙选切或不切，统计该数字的全部切法。
for (int mask = 0; mask < (1 << (length - 1)); mask++) {
    int sum = 0, value = 0, cuts = 0;
    for (int j = 0; j < length; j++) {
        value = (value * 10 + s[j] - '0') % k;
        if (j == length - 1 || (mask & (1 << j)) != 0) {
            sum = (sum + value) % k;
            value = 0;
            if (j != length - 1) {
                cuts++;
            }
        }
    }
    best[sum] = min(best[sum], cuts);
}
for (int r = 0; r < k; r++) {
    if (dp[r] == INF) {
        continue;
    }
    for (int t = 0; t < k; t++) {
        if (best[t] != INF) {
            int next = (r + t) % k;
            nextDp[next] = min(nextDp[next], dp[r] + best[t]);
        }
    }
}
for (int r = 0; r < k; r++) {
    dp[r] = nextDp[r];
}
}
if (dp[0] == INF) {
    cout << -1 << '\n';
} else {
    cout << dp[0] << '\n';
}
return 0;
}

```

满分做法：数位位置与余数动态规划

不必枚举一个数的全部切法，可以改为逐段决定“下一段在哪里结束”。

将所有原数的数位顺次存入字符串，同时记录 `rightEnd[pos]`：位置 `pos` 所属原数的最后一个数位在哪里。这样便于统一枚举，但任何一段都不能跨过这个右端点。

令 $dp[i][r]$ 表示：前 i 个数位已经分段完毕，各段数值之和模 k 为 r 时，最少需要的切割次数。初始 $dp[0][0] = 0$ ，其余状态不可达。

从状态 $dp[i][r]$ 出发，下一段从 $i + 1$ 开始。枚举右端 j ，范围为

$$i + 1 \leq j \leq \text{rightEnd}[i + 1].$$

向右延长时，逐位维护这一段的余数 `value`。如果 j 尚未到原数末尾，就需要在 j 后面切一次，令 $\text{cost} = 1$ ；如果 j 恰好在原数末尾，这里本来就分属两个原数，不必切，令 $\text{cost} = 0$ 。

于是转移为

$$dp[j][(r + value) \bmod k] = \min(dp[j][(r + value) \bmod k], dp[i][r] + cost).$$

任意合法切法都能按顺序拆成这样的逐段选择；每个内部切点恰好在前一段结束时计费一次。因此，所有数位处理完成后，余数为 0 的最小代价就是答案。完全不切也可能最优；该状态不可达时才输出 -1。

设第 t 个数的长度为 L_t ，总长度为 $L = \sum_t L_t$ 。时间复杂度为 $O(k \sum_t L_t^2)$ ，空间复杂度为 $O(Lk)$ 。 10^{18} 有 19 位；只维护余数可以避免分段数值相加时溢出。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1000000000;
int dp[8005][405], rightEnd[8005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, k;
    cin >> n >> k;
    string s = " ";
    for (int i = 1; i <= n; i++) {
        string t;
        cin >> t;
        int start = (int)s.size();
        s += t;
        int finish = (int)s.size() - 1;
        for (int j = start; j <= finish; j++) {
            rightEnd[j] = finish;
        }
    }
    int length = (int)s.size() - 1;
    for (int i = 0; i <= length; i++) {
        for (int r = 0; r < k; r++) {
            dp[i][r] = INF;
        }
    }
    dp[0][0] = 0;
    // dp[i][r]: 前 i 位已分段完成，数字和模 k 为 r 的最少切割数。
    for (int i = 0; i < length; i++) {
        int value = 0;
        for (int j = i + 1; j <= rightEnd[i + 1]; j++) {
            value = (value * 10 + s[j] - '0') % k;
            int cost = 0;
            if (j < rightEnd[i + 1]) {
                cost = 1;
            }
            for (int r = 0; r < k; r++) {
                if (dp[i][r] == INF) {
                    continue;
                }
                int next = (r + value) % k;
                dp[j][next] = min(dp[j][next], dp[i][r] + cost);
            }
        }
    }
    if (dp[length][0] == INF) {
        cout << -1 << '\n';
    } else {
        cout << dp[length][0] << '\n';
    }
    return 0;
}
```